SCRIPT DE DESCARGA · ETAPA 0

00_descarga_faostat.py

Descarga automática de los 3 datasets FAOSTAT · Aguacate Team · TFM "La Dieta de un País" · 2026

TIPO	OUTPUT	TAMAÑO TOTAL DESCARGA	ESTADO
Script Python (idempotente)	**`raw/` — 3 CSVs**	**~986 MB**	✅ **Ejecutado**

DATASETS DESCARGADOS

1 — Food Balance Sheets (FBS)

Eje	**Nutrición** — Dietary Energy Supply (kcal/cápita/día)
Destino	`raw/fbs_raw.csv` — 604 MB
ZIP	`FoodBalanceSheets_E_All_Data_(Normalized).zip`
URL	`bulks-faostat.fao.org/production/FoodBalanceSheets_E_All_Data_(Normalized).zip`

2 — Emissions Agrifood System

Eje	**Clima** — CO_2eq toneladas/cápita/año del sistema agroalimentario
Destino	`raw/emissions_raw.csv` — 347 MB
ZIP	`Emissions_Totals_E_All_Data_(Normalized).zip`
URL	`bulks-faostat.fao.org/production/Emissions_Totals_E_All_Data_(Normalized).zip`

3 — Consumer Price Indices (Food CPI)

Eje	**Coste** — Índice de precios alimentarios por país (base 2015=100)
Destino	`raw/cpi_raw.csv` — 35.5 MB
ZIP	`ConsumerPriceIndices_E_All_Data_(Normalized).zip`
URL	`bulks-faostat.fao.org/production/ConsumerPriceIndices_E_All_Data_(Normalized).zip`

LÓGICA DE EJECUCIÓN

Función `descargar(dataset)` — flujo por dataset

1. **Idempotencia:** si el CSV destino ya existe en `raw/`, lo omite sin re-descargar. El script es re-ejecutable sin efectos secundarios.
2. **Descarga en streaming** con chunks de 64 KB (`stream=True`) para no cargar el ZIP completo en RAM. Muestra progreso en porcentaje.
3. **Extracción ZIP en memoria** — el ZIP se descomprime desde `io.BytesIO` sin escribir el ZIP en disco. Solo se escribe el CSV final.
4. **Búsqueda flexible del CSV** dentro del ZIP — busca por nombre exacto o por extensión `.csv` como fallback, por si FAOSTAT cambia la estructura interna del ZIP.
5. **Manejo de errores** — captura `RequestException` (red) y `BadZipFile` (descarga corrupta) por separado. Devuelve `True/False` por dataset.
6. **Resumen final** — imprime estado de cada dataset (✓ / ✗ FALLO) tras procesar los tres.

> **Cómo re-ejecutar:** `python notebooks/00_descarga_faostat.py` desde la raíz del proyecto. Si los 3 CSVs ya existen, termina en segundos sin tocar nada. Si alguno falta, lo descarga solo.

CÓDIGO FUENTE COMPLETO

00_descarga_faostat.py — Python 3 · 110 líneas

```python
# -*- coding: utf-8 -*-
"""
00_descarga_faostat.py
Descarga automática de los 3 datasets FAOSTAT para el TFM "La Dieta de un País"
Aguacate Team · 2026
"""

import requests
import zipfile
import io
import os

RAW_DIR = os.path.join(os.path.dirname(__file__), '..', 'raw')
os.makedirs(RAW_DIR, exist_ok=True)

DATASETS = [
    {
        "nombre": "Food Balance Sheets",
        "url": "https://bulks-faostat.fao.org/production/FoodBalanceSheets_E_All_Data_(Normalized).zip",
        "archivo_zip": "FoodBalanceSheets_E_All_Data_(Normalized).csv",
        "destino": "fbs_raw.csv"
    },
    {
        "nombre": "Emissions Agrifood System",
        "url": "https://bulks-faostat.fao.org/production/Emissions_Totals_E_All_Data_(Normalized).zip",
        "archivo_zip": "Emissions_Totals_E_All_Data_(Normalized).csv",
        "destino": "emissions_raw.csv"
    },
    {
        "nombre": "Consumer Price Indices (Food CPI)",
        "url": "https://bulks-faostat.fao.org/production/ConsumerPriceIndices_E_All_Data_(Normalized).zip",
        "archivo_zip": "ConsumerPriceIndices_E_All_Data_(Normalized).csv",
        "destino": "cpi_raw.csv"
    }
]

def descargar(dataset):
    nombre  = dataset["nombre"]
    url     = dataset["url"]
    destino = os.path.join(RAW_DIR, dataset["destino"])

    if os.path.exists(destino):
        size_mb = os.path.getsize(destino) / 1_048_576
        print(f"[✓] {nombre} ya existe ({size_mb:.1f} MB) — omitiendo.")
        return True

    print(f"\n[→] Descargando: {nombre}")
    print(f"    URL: {url}")

    try:
        r = requests.get(url, timeout=120, stream=True)
        r.raise_for_status()

        total      = int(r.headers.get('content-length', 0))
        descargado = 0
        chunks     = []

        for chunk in r.iter_content(chunk_size=65536):
            chunks.append(chunk)
            descargado += len(chunk)
            if total:
                pct = descargado / total * 100
                print(f"\r    Progreso: {pct:.1f}% ({descargado/1_048_576:.1f} MB)", end='', flush=True)

        print()
        contenido = b''.join(chunks)

        with zipfile.ZipFile(io.BytesIO(contenido)) as z:
            nombres = z.namelist()
            print(f"    Archivos en ZIP: {nombres}")

            # Buscar el CSV correcto dentro del ZIP
            csv_target = dataset["archivo_zip"]
            match = next((n for n in nombres if csv_target in n or n.endswith('.csv')), None)

            if not match:
                print(f"[!] No se encontró CSV en el ZIP. Archivos: {nombres}")
                return False

            print(f"    Extrayendo: {match}")
            with z.open(match) as f_in, open(destino, 'wb') as f_out:
                f_out.write(f_in.read())

        size_mb = os.path.getsize(destino) / 1_048_576
        print(f"[✓] {nombre} guardado → raw/{dataset['destino']} ({size_mb:.1f} MB)")
        return True

    except requests.exceptions.RequestException as e:
        print(f"[✗] Error descargando {nombre}: {e}")
        return False
    except zipfile.BadZipFile as e:
        print(f"[✗] ZIP corrupto para {nombre}: {e}")
        return False

if __name__ == "__main__":
    print("=" * 60)
    print("FAOSTAT Downloader — Aguacate Team TFM 2026")
    print("=" * 60)
    resultados = []
    for ds in DATASETS:
        ok = descargar(ds)
        resultados.append((ds["nombre"], ok))

    print("\n" + "=" * 60)
    print("RESUMEN:")
    for nombre, ok in resultados:
        estado = "✓" if ok else "✗ FALLO"
        print(f"  [{estado}] {nombre}")
    print("=" * 60)
```